FACTS

FACTS Case Management System

Business Requirements Document

VERSION	DATE	STATUS
3.7	April 23, 2025	Final

• LIVE & OPERATIONAL

FACTS Case Management System

Business Requirements Document (BRD)

Document Version: 3.7
Date: April 23, 2025
Prepared By: FACTS Development Team
Document Status: Final
Classification: Internal

Document Control

ITEM	DETAIL
Project Name	FACTS — File and Case Tracking System
Document Type	Business Requirements Document (BRD)
Version	3.7
Date	April 23, 2025
Organizations Served	Baaseteen & SHND (Social Help and Development)
Current Status	Live & Operational

Table of Contents

1. Executive Summary
2. Business Objectives
3. Project Scope
4. Stakeholders
5. System Overview
6. User Roles and Permissions
7. Functional Requirements
    - 7.1 Authentication & Authorization Module
    - 7.2 User Management Module
    - 7.3 Case Identification Module
    - 7.4 Case Management Module
    - 7.5 Applicant Management Module
    - 7.6 Counseling Forms Module
    - 7.7 Cover Letter Module
    - 7.8 Payment Management Module
    - 7.9 Fund Utilization Module
    - 7.10 Welfare Checklist Module
    - 7.11 Workflow & Approval Module
    - 7.12 Dashboard & Analytics Module
    - 7.13 Notification System Module
    - 7.14 Reporting Module
8. Non-Functional Requirements
9. System Architecture
10. Database Design
11. User Interface Requirements
12. Data Flow Diagrams
13. Module Dependencies
14. Integration Points
15. Deployment Architecture
16. Security Requirements
17. Performance Requirements
18. Assumptions and Constraints
19. Risks and Mitigation
20. Acceptance Criteria
21. Glossary
22. Appendices

---

1. Executive Summary

1.1 Purpose

This Business Requirements Document defines the complete functional and non-functional requirements for the **FACTS Case Management System** — a comprehensive full-stack web application designed to manage the entire lifecycle of welfare cases for the Baaseteen and SHND (Social Help and Development) organizations. FACTS digitizes and centralizes every stage of welfare case operations, from applicant identification and eligibility assessment through multi-level approval workflows, financial disbursement, and quarterly fund utilization tracking.

1.2 Background

Prior to FACTS, administrative teams at Baaseteen and SHND relied on deeply decentralized, manual welfare review mechanisms. Paper-based processes led to prolonged service-level turnaround times, opaque executive approval chains, delays in financial disbursements to beneficiaries, and unreliable quarterly performance assessments. The absence of a unified tracking platform made audit trails incomplete and reporting cumbersome. FACTS was commissioned to replace these manual processes with a centralized digital platform capable of enforcing structured, transparent, and auditable workflows across geographically distributed teams.

1.3 Solution Overview

FACTS is a three-tier web application built on Node.js (Express), React, and MySQL, providing:

- **End-to-end case lifecycle management** from pre-screening through closure, with configurable multi-stage approval pipelines
- **ITS (Institutional Information System) integration** for automated applicant identity retrieval and photo verification
- **Eight-section digital counseling forms** replacing paper-based counseling documentation
- **Multi-level executive approval hierarchy** with up to 5 approval levels, rework and rejection flows
- **Payment schedule management** tracking Qardan Hasana and Enayat disbursements with installment tracking
- **Dynamic fund utilization reporting** with a configurable form builder for quarterly reassessments
- **Automated PDF generation** for cover letters and quarterly reports via PDFKit and Puppeteer
- **Granular Role-Based Access Control (RBAC)** with Jamiat/Jamaat organizational scope isolation
- **Real-time in-app notifications** for case status changes, assignments, and SLA alerts
- **SLA tracking** with automated breach detection and warning alerts per workflow stage

1.4 Key Metrics

METRIC	DETAIL
Active User Roles	8 (Super Admin, Admin, DCM, Counselor, Welfare Reviewer, Executive, Finance, Report Viewer)
Core Modules	14
Database Tables	50+ (normalized relational schema)
API Endpoints	150+ RESTful endpoints across 25 route files
Workflow Stages	Up to 10 configurable stages per case type
Counseling Form Sections	8 (Personal Details, Family Details, Assessment, Financial Assistance, Economic Growth, Declaration, Attachments, Manzoori)
Tech Stack	Node.js + Express / React 18 / MySQL 8.0
PDF Generation	PDFKit + Puppeteer (cover letters & quarterly reports)
External Integrations	ITS API (applicant auto-fetch), SMTP email, Excel import/export

2. Business Objectives

2.1 Primary Objectives

ID	OBJECTIVE	PRIORITY
BO-1	Digitize the complete welfare case lifecycle — from applicant identification and eligibility screening through counseling, multi-level approvals, disbursement, and fund utilization tracking — eliminating paper-based silos	Critical
BO-2	Enforce transparent, configurable multi-tier workflow approval pipelines that prevent cases from stagnating at any stage through automated SLA monitoring	Critical
BO-3	Automate applicant data acquisition through ITS system integration, reducing manual data entry errors and accelerating case intake	High
BO-4	Standardize counseling documentation through structured 8-section digital forms with digital signature capture and file attachment support	High
BO-5	Automate cover letter and quarterly fund utilization report generation with PDF export, removing manual document preparation bottlenecks	High
BO-6	Enforce organizational access boundaries through Jamiat/Jamaat-scoped RBAC, ensuring each user only operates within their authorized geographic jurisdiction	Critical
BO-7	Provide executive and administrative stakeholders with real-time dashboard analytics and pipeline visibility across all active cases	Medium
BO-8	Track financial disbursements (Qardan Hasana and Enayat) with quarterly repayment schedules and confirmation workflows	High

2.2 Success Criteria

- All welfare cases are tracked digitally from case identification through closure with complete audit trail
- Applicant data can be auto-populated from the external ITS API by entering an ITS number
- Cases progress through configurable workflow stages with role-enforced gate approvals
- SLA breach alerts are triggered automatically when cases exceed defined time thresholds per stage
- Counseling forms are completed entirely within the system with digital signatures
- Cover letters and quarterly reports can be generated and downloaded as PDFs with one click
- Payment schedules track disbursements and repayment timelines per case
- Users in different Jamiats/Jamaats cannot access each other's case data
- System is available 24/7 on the production server

3. Project Scope

3.1 In Scope

AREA	DESCRIPTION
Case Identification	Pre-assessment eligibility screening with ITS integration, income assessment, and Jamiat/Jamaat filtering
Case Management	Full case lifecycle with case number generation, status tracking, assignment, comments, audit trail, and closure
Applicant Management	Comprehensive profiles with personal, family, education, occupation details; ITS auto-fetch; bulk import from Excel
Counseling Forms	8-section digital counseling forms covering personal details, family details, assessment, financial assistance, economic growth, declaration, attachments, and Manzoori (approval)
Cover Letters	Automated cover letter form workflow with PDF generation and download
Payment Management	Qardan Hasana and Enayat payment schedules with quarterly/yearly installments and disbursement confirmation
Fund Utilization	Dynamic quarterly reassessment reports with configurable form builder (admin-defined sections and question types), photo documentation, PDF export
Welfare Checklist	Configurable checklist categories and items with drag-and-drop reordering and per-case completion tracking
Workflow & Approvals	Configurable multi-stage approval pipelines per case type; approve, reject, rework, resubmit actions with comments
SLA Tracking	Per-stage SLA configuration with warning and breach detection; automated SLA check triggering
RBAC	Granular resource + action permission model with Jamiat/Jamaat scope isolation; multi-role user support
Notifications	In-app notifications for case status changes, assignments, and SLA alerts; unread badge and history
Dashboard Analytics	Role-specific statistics, pipeline views, and recent activity feeds
Reporting	PDF generation for cover letters and fund utilization reports; Excel import/export for bulk operations
File Attachments	Multi-file upload per case and per counseling section; type and size validation; download capability
Master Data	Case types, relations, education levels, occupations, executive levels, workflow stages, Jamiat, Jamaat
Deployment	PM2 / systemd production deployment with Nginx reverse proxy; SSL support

3.2 Out of Scope

AREA	REASON
Mobile Native App	Responsive web design serves mobile needs; native app is a future consideration
Payment Gateway Integration	Financial disbursements are tracked in the system but processed externally; PCI compliance is out of scope
Public Applicant Portal	Applicants do not self-register; all intake is performed by authorized counselors/DCMs
Multi-Tenancy	Single-deployment for Baaseteen/SHND; SaaS multi-tenancy not required
Video Conferencing	External tools used for remote counseling sessions
SMS Notifications	Current system uses in-app and email notifications only
Biometric Authentication	Standard username/password + JWT; MFA is a future roadmap item

AREA	REASON
Public API	Internal use only; external API is a future roadmap item

4. Stakeholders

4.1 Stakeholder Matrix

STAKEHOLDER	ROLE	INTEREST	INFLUENCE
Baaseteen / SHND Administration	Project Sponsor	High — System aligns with organizational welfare delivery mandate	High
System Administrators	Primary Users (Admin Role)	High — Manage all configurations, users, and master data	High
DCMs (Deputy Case Managers)	Primary Users	High — Assign counselors, oversee case progress, manage counseling	High
Counselors	Primary Users	High — Daily use for counseling form completion and case management	Medium
Welfare Reviewers	Primary Users	High — Review and approve/reject cases post-counseling	High
Executive Management	Secondary Users	Medium — Multi-level approval of sanctioned welfare cases	High
Finance Team	Secondary Users	Medium — Track and confirm financial disbursements	Medium
Report Viewers	Secondary Users	Low — Read-only access for oversight and reporting	Low
Development Team	System Builders	High — Technical implementation and maintenance	High

4.2 Target Users

USER TYPE	DESCRIPTION	ESTIMATED COUNT
Super Admin / Admin	System managers who configure users, master data, roles, and system-wide settings	2–5
DCMs	Deputy Case Managers who assign counselors, track cases, and manage workflow progression	5–15
Counselors	Field workers who conduct applicant assessments and complete counseling documentation	20–50
Welfare Reviewers	Welfare department members who review counseling completions and approve/reject cases	5–10
Executive Management	Senior leadership performing staged financial sanction approvals	5–15
Finance Team	Finance staff confirming disbursements and managing payment records	3–8
Report Viewers	Stakeholders with read-only access to reports and dashboards	5–10

5. System Overview

5.1 Technology Stack

LAYER	TECHNOLOGY	VERSION
Backend Runtime	Node.js + Express.js	v16+ / v4.18
Frontend Framework	React (functional components + hooks)	v18.2
Database	MySQL (InnoDB)	v8.0+
Authentication	JWT + bcryptjs	jsonwebtoken v9
State Management	React Query (server state)	v3.39
Form Handling	React Hook Form	v7.48
Styling	Tailwind CSS	v3.4
HTTP Client	Axios	v1.6
Routing	React Router	v6
Charts	Recharts	v2.8
File Upload	Multer	v1.4
Email	Nodemailer	v6.9
PDF Generation	PDFKit + Puppeteer	v0.14 / v24
Excel Handling	xlsx	v0.18
Input Validation	Joi	v17.11
Security	Helmet, CORS, express-rate-limit	Latest
Drag & Drop	@dnd-kit	Latest
Signatures	Custom SignaturePad component	Internal
Web Server / Proxy	Nginx (reverse proxy)	Latest stable
Process Manager	PM2 / systemd	Latest

5.2 Architecture Pattern

FACTS follows a **decoupled three-tier architecture**:

- **Client Tier**: React 18 Single Page Application (SPA) consuming a stateless JSON REST API. All data is managed via React Query with automatic caching and optimistic updates. Auth state is managed by a global React Context (`AuthContext`).
- **API Gateway Tier**: Node.js / Express server exposing 25 route modules with ~150+ endpoints. JWT middleware enforces authentication; a RBAC middleware enforces resource-level permissions and Jamiat/Jamaat scope on every request. Business logic is encapsulated in service and utility files.
- **Data Tier**: MySQL 8.0 InnoDB database with 50+ normalized tables, enforced foreign key constraints, and parameterized queries throughout. File uploads are stored on local filesystem under `uploads/` .

5.3 High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                CLIENT LAYER                              │
│  ┌──────────────────────┐  ┌────────────────┐  ┌──────────────┐        │
│  │ React 18 SPA         │  │ React Query    │  │ React Router │  │
│  │ Tailwind CSS         │  │ (Server State) │  │ v6 (Routing) │  │
│  │ 31 Page Components   │  │ Axios HTTP     │  │ ProtectedRoute│  │
│  │ 16 UI Components     │  │ React Hook Form│  │ (Auth Guard) │  │
│  └──────────┬───────────┘  └────────┬───────┘  └──────┬───────┘  │
└─────────────┼──────────────────────┼────────────────┼──────────┘
              │                      │                │
              │          Stateless JSON / REST API    │
              ▼                      ▼                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                        API GATEWAY LAYER (Node.js/Express)               │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  auth.js Middleware: JWT Verification + User Status Check        │  │
│  │  permissionMiddleware.js: Resource:Action + Scope Filtering      │  │
│  ├──────────────────────────────────────────────────────────────────┤  │
│  │  Route Modules (25 files):                                       │  │
│  │  auth · users · cases · applicants · caseIdentifications         │  │
│  │  counselingForms · coverLetters · coverLetterForms               │  │
│  │  fundUtilization · fundUtilizationMaster · quarterlyReportPdf    │  │
│  │  dashboard · notifications · attachments · workflowStages        │  │
│  │  welfareChecklist · roles · permissions · jamiat · jamaat        │  │
│  │  caseTypes · relations · educationLevels · occupations           │  │
│  │  executiveLevels · businessAssets                                │  │
│  ├──────────────────────────────────────────────────────────────────┤  │
│  │  Services:                                                       │  │
│  │  emailService.js · notificationService.js                        │  │
│  ├──────────────────────────────────────────────────────────────────┤  │
│  │  Utils:                                                          │  │
│  │  slaCalculator.js · activeCasePerIts.js · permissionUtils        │  │
│  │  roleUtils.js · quarterlyReportPdfData · PdfKit builder          │  │
│  └──────────────────────────────────────────────────────────────────┘  │
└──────────────────────────┬──────────────────────────────────────────┘
                           │
          ┌────────────────┴────────────────┐
          ▼                                 ▼
┌────────────────────┐          ┌──────────────────────────────┐
│  DATA LAYER        │          │  FILE STORAGE                │
│  MySQL 8.0 InnoDB  │          │  uploads/                    │
│  50+ tables        │          │  ├── (case attachments)      │
│  FK constraints    │          │  ├── fund utilization photos │
│  Parameterized SQL │          │  └── fund utilization files  │
└────────────────────┘          └──────────────────────────────┘
```

6. User Roles and Permissions

6.1 Role Definitions

6.1.1 Super Admin

Description: Unrestricted system access. Manages roles, permissions, all users, all cases, all master data, and system configuration. No Jamiat/Jamaat scope restrictions.

PERMISSION AREA	ACCESS LEVEL
User Management	Full CRUD — all users, all roles, executive level assignment
Role & Permission Management	Create, edit, delete roles; assign permissions; manage counseling form stage permissions
All Case Operations	View, create, update, delete, approve, reject, rework, close — any case
Master Data	Full CRUD — case types, Jamiat, Jamaat, relations, education levels, occupations, executive levels, workflow stages
Reports & Dashboard	Full access to all analytics and all batch exports
System Settings	Full access to system configuration

6.1.2 Admin

Description: Administrative functions without full role management. Manages users, master data, case oversight, and reporting.

6.1.3 DCM (Deputy Case Manager)

Description: Jamiat/Jamaat-scoped access. Manages case intake, assigns counselors, oversees counseling completion, and manages workflow progression for cases within their assigned organizational scope.

PERMISSION	ACCESS
Case CRUD	Create, read, update cases within assigned Jamiat/Jamaat scope
Counselor Assignment	Assign and reassign counselors to cases
Counseling Form	View counseling forms; submit to welfare review
Cover Letter	Generate and manage cover letters
Payment Schedules	Create and manage payment schedules
Reports	View reports for assigned scope

6.1.4 Counselor

Description: Jamaat-scoped access. Fills multi-section counseling forms, uploads supporting documents, and captures digital signatures for assigned cases.

PERMISSION	ACCESS
Counseling Form	Edit all 8 sections for assigned cases; upload attachments per section
Case View	Read-only access to assigned case details
Applicant Data	View applicant profiles; ITS lookup

6.1.5 Welfare Reviewer

Description: Jamiat-scoped access. Reviews completed counseling forms, approves or rejects cases for ZI progression, completes welfare checklists.

PERMISSION	ACCESS
Case Approval	Approve or reject cases within assigned Jamiat scope

PERMISSION	ACCESS
Welfare Checklist	Complete and review checklist responses per case
Case Comments	Add welfare-level comments and feedback
Cover Letter Form	Update and submit cover letter form fields

6.1.6 Executive Management

Description: Multi-level approval authority. Each executive user is assigned to a specific executive level (Level 1–5). Cases requiring executive sanction progress sequentially through each assigned level.

PERMISSION	ACCESS
Executive Approval	Approve or request rework for cases at their assigned executive level
Case Comments	Add executive-level comments (optionally visible/hidden from DCM)
Dashboard	View pending executive approval queue

6.1.7 Finance

Description: Manages financial disbursement confirmation for approved cases. Confirms actual disbursement dates and amounts for payment schedule installments.

6.1.8 Report Viewer

Description: Read-only access limited to reports and dashboard analytics. Cannot create or modify any records.

6.2 Role Assignment Matrix

FEATURE	SUPER ADMIN	ADMIN	DCM	COUNSELOR	WELFARE REVIEWER	EXECUTIVE	FINANCE
Manage Users	✓	✓	✗	✗	✗	✗	✗
Manage Roles	✓	✗	✗	✗	✗	✗	✗
Create Cases	✓	✓	✓*	✗	✗	✗	✗
Assign Counselors	✓	✓	✓*	✗	✗	✗	✗
Fill Counseling Forms	✓	✓	✓	✓*	✗	✗	✗
Welfare Approve/Reject	✓	✓	✗	✗	✓*	✗	✗
ZI Approve/Reject	✓	✓	✓	✗	✗	✗	✗
Executive Approve	✓	✓	✗	✗	✗	✓†	✗
Generate Cover Letter	✓	✓	✓*	✗	✗	✗	✗
Confirm Disbursement	✓	✓	✓	✗	✗	✗	✓
Fund Utilization Reports	✓	✓	✓	✗	✗	✗	✓
Manage Master Data	✓	✓	✗	✗	✗	✗	✗
View Dashboard	✓	✓	✓	✓	✓	✓	✓
Close Case	✓	✓	✓*	✗	✗	✗	✗

** Restricted to assigned Jamiat/Jamaat scope* *† Restricted to assigned executive level only*

7. Functional Requirements

7.1 Authentication & Authorization Module

FR-AUTH-001: User Login

FIELD	DETAIL
Priority	Critical
Description	The system shall authenticate users via username/email and password, returning a signed JWT token valid for 24 hours
Acceptance Criteria	User can log in with username or email; incorrect credentials return 401; successful login returns JWT + user object with role and permissions; token is validated on every subsequent protected API request

Business Rules:

- Passwords are stored as bcrypt hashes; `password_hash()` used at creation, `bcrypt.compare()` at login
- JWT payload contains `userId`, `username`, `role`, and `executive_level`
- User `is_active` flag is checked on every authenticated request — inactive users are rejected mid-session
- Login endpoint is rate-limited to prevent brute-force attacks
- No self-service password reset; admin must reset passwords

FR-AUTH-002: JWT Session Management

FIELD	DETAIL
Priority	Critical
Description	The system shall enforce JWT-based stateless sessions for all protected API routes
Acceptance Criteria	Requests without a valid Bearer token receive 401; expired tokens receive 401; logout is client-side token discard; profile and permissions retrievable from `GET /api/auth/profile`

FR-AUTH-003: Role-Based Access Control

FIELD	DETAIL
Priority	Critical
Description	The system shall enforce granular RBAC at the API route level using a `resource:action` permission model
Acceptance Criteria	Each protected endpoint checks the requesting user's role permissions against the required resource:action pair; insufficient permissions return 403; Super Admin bypasses permission checks; permissions are configurable per role via admin UI

FR-AUTH-004: Jamiat/Jamaat Scope Isolation

FIELD	DETAIL
Priority	Critical
Description	The system shall filter data results based on the authenticated user's assigned Jamiat/Jamaat organizational scope
Acceptance Criteria	Approval roles (Welfare, ZI) see only cases from their assigned Jamiat(s); filling roles (Counselors) see only cases from their assigned Jamaat(s); Super Admin and Admin have no scope restriction; scope is enforced at the database query level, not just UI filtering

FR-AUTH-005: Multi-Role User Support

FIELD	DETAIL
Priority	High
Description	Users may be assigned multiple roles with different Jamiat/Jamaat scopes per role

FIELD	DETAIL
Acceptance Criteria	Roles stored in `user_roles` junction table with per-role Jamiat/Jamaat assignments; user can switch primary display role via `PATCH /api/users/me/primary-role` ; system resolves effective permissions from all assigned roles

---

7.2 User Management Module

FR-USER-001: Create User

FIELD	DETAIL
Priority	Critical
Description	Admins shall create user accounts with full profile, role assignment, and optional organizational scope
Acceptance Criteria	Required fields: full_name, username, email, role; optional: phone, its_number, photo (base64), Jamiat/Jamaat scope, role_jamiat_jamaat mapping; username and email uniqueness enforced; password defaults to `TempPassword123!` if not provided; user can be assigned multiple roles with per-role scope at creation

FR-USER-002: Update User

FIELD	DETAIL
Priority	High
Description	Admins shall update all user profile fields, role assignments, and organizational scope
Acceptance Criteria	All fields updatable; password change optional; role and scope reassignment supported; uniqueness constraints enforced on update

FR-USER-003: Activate / Deactivate User

FIELD	DETAIL
Priority	High
Description	Admins shall activate or deactivate user accounts without deleting them
Acceptance Criteria	Toggle endpoint changes `is_active` flag; deactivated users are denied API access mid-session; account data is preserved

FR-USER-004: List and Filter Users

FIELD	DETAIL
Priority	High
Description	Admins shall view a paginated, filterable list of all users
Acceptance Criteria	Filters: role, is_active, search (name/email/username), jamiat_id; pagination with configurable limit (10/20/50/100/500); response includes assigned_roles with per-role scope; total count returned for pagination controls

FR-USER-005: Assign Executive Level

FIELD	DETAIL
Priority	High
Description	Admins shall assign an executive level number to users who hold executive approval authority
Acceptance Criteria	Executive level stored in `users.executive_level` ; level references configured levels in `executive_levels` table; affects which cases appear in the user's executive approval queue

FR-USER-006: Bulk Import and Export

FIELD	DETAIL
Priority	Medium
Description	Admins shall import users in bulk via Excel file and export user lists to Excel
Acceptance Criteria	Excel template downloadable from `GET /api/users/export/template` ; import returns row-level results (inserted/updated/skipped/errors); ITS API fetch available for users with ITS numbers via `POST /api/users/fetch-contact-from-api`

7.3 Case Identification Module

FR-CASEID-001: Create Case Identification

FIELD	DETAIL
Priority	Critical
Description	Authorized users shall create pre-assessment eligibility screening records for potential welfare applicants
Acceptance Criteria	Required: its_number, eligible_in (case type), total_family_members (0–25), earning_family_members (0–20), individual_income (0–9,999,999), family_income (0–9,999,999); applicant data auto-populated via ITS number lookup; duplicate ITS check blocks re-registration; status defaults to 'pending'

FR-CASEID-002: ITS Number Auto-Lookup

FIELD	DETAIL
Priority	Critical
Description	The system shall auto-populate applicant demographic data by querying the external ITS API when an ITS number is entered
Acceptance Criteria	Fields populated: full_name, age, gender, phone, email, photo (base64), address, jamiat, jamaat; lookup first checks local DB (`applicants` table), then falls back to external ITS API; failure returns graceful empty response; existing ITS numbers flagged for duplicate detection

FR-CASEID-003: Review — Eligible / Ineligible

FIELD	DETAIL
Priority	Critical
Description	Authorized reviewers shall mark case identification records as eligible or ineligible
Acceptance Criteria	Permission: `case_identification:approve` ; marking 'eligible' auto-creates a Case record with an auto-generated case number (e.g., BS-0001); marking 'ineligible' stores review_remarks and closes the identification; reviewer and timestamp recorded; notification sent on status change

FR-CASEID-004: Edit Assessment Fields

FIELD	DETAIL
Priority	High
Description	Authorized editors shall modify income and assessment fields on pending or ineligible case identification records
Acceptance Criteria	Permission: `case_identification:edit` ; editable fields: individual_income, family_income, eligible_in, total_family_members, earning_family_members, remarks; all income field changes logged to `case_identification_income_logs` with old/new values and editor ID for full audit trail

FR-CASEID-005: Revert Ineligible to Pending

FIELD	DETAIL
Priority	Medium
Description	Authorized editors shall revert an ineligible case identification back to pending status for re-review
Acceptance Criteria	Only ineligible records can be reverted; status changes to 'pending'; review fields (reviewed_by, reviewed_at, review_remarks) cleared; audit log entry created

---

7.4 Case Management Module

FR-CASE-001: Create Case

FIELD	DETAIL
Priority	Critical
Description	Cases shall be created either automatically from an eligible case identification or manually by an authorized user
Acceptance Criteria	Auto-creation: triggered on 'eligible' review decision; case number generated (e.g., BS-0001, sequential); linked to applicant record; Manual creation: requires applicant_id or inline applicant_data; requires case_type_id; optional: DCM assignment, counselor assignment, Jamiat/Jamaat override; a counseling form record is auto-created for each new case

FR-CASE-002: Case Assignment

FIELD	DETAIL
Priority	Critical
Description	DCMs/Admins shall assign cases to counselors with Jamiat/Jamaat scope filtering
Acceptance Criteria	Available counselors endpoint filters by role and optional Jamiat/Jamaat; assignment stored in `cases.assigned_counselor_id`; DCM stored in `cases.roles`; notification sent to assigned counselor on assignment

FR-CASE-003: Case Workflow Actions

FIELD	DETAIL
Priority	Critical
Description	Cases shall progress through configurable workflow stages via approve, reject, rework, and resubmit actions
Acceptance Criteria	Each action validates user's role permission against the current workflow stage; approve advances case to next stage; reject records reason and blocks case; rework returns case to previous stage; resubmit re-advances a reworked case; all actions write to `status_history` and `workflow_logs`; stage entry timestamps recorded in `cases.current_stage_entered_at` for SLA calculation

FR-CASE-004: Case Closure

FIELD	DETAIL
Priority	High
Description	Authorized users shall close completed cases by uploading closure documentation
Acceptance Criteria	Permission: `cases:close_case`; requires closure reason and 1–10 file uploads (PDF, DOC, DOCX, XLS, XLSX, JPG, PNG; max 10MB each); creates `case_closures` and `case_closure_documents` records; case status set to 'closed'

FR-CASE-005: Case Comments

FIELD	DETAIL
Priority	High
Description	Authorized users shall add general and workflow-specific comments to cases
Acceptance Criteria	General comments stored in `case_comments`; workflow-step-specific comments stored in `workflow_comments`; executive comments support `is_visible_to_dcm` flag and `executive_level` tag; comment type options: general, rejection, approval, note, feedback

FR-CASE-006: Duplicate ITS Detection

FIELD	DETAIL
Priority	High
Description	The system shall enforce one active case per ITS number and flag duplicates
Acceptance Criteria	Check: `GET /api/cases/its/:itsNumber/active` returns active cases for the ITS; if an active case exists, new case creation is blocked or flagged with `is_duplicate = 1`; duplicate cases shown in case list with visual flag

FR-CASE-007: Case Filtering and Pagination

FIELD	DETAIL
Priority	High
Description	Users shall filter and paginate case lists based on multiple criteria
Acceptance Criteria	Filter parameters: status, case_type, case_type_id, assigned DCM, assigned counselor, search (name/ITS/case number), jamiat_id, jamaat_id, current_workflow_stage_id, is_duplicate; results scoped by user's Jamiat/Jamaat assignment; SLA status (on_time/at_risk/breached) calculated and returned per case

---

7.5 Applicant Management Module

FR-APP-001: Applicant CRUD

FIELD	DETAIL
Priority	High
Description	Authorized users shall manage comprehensive applicant profiles
Acceptance Criteria	Fields: its_number, full_name, age, gender, phone, email, photo (base64), address, jamiat_name, jamaat_name, jamiat_id, jamaat_id; ITS number uniqueness enforced; optional inline case creation on applicant creation

FR-APP-002: ITS Auto-Fetch

FIELD	DETAIL
Priority	Critical
Description	The system shall fetch applicant data from the external ITS API by ITS number
Acceptance Criteria	`GET /api/applicants/fetch-from-api/:itsNumber` retrieves full_name, age, gender, phone, email, photo, address, Jamiat/Jamaat, country, city, state, occupation, qualification, idara; returned data used to pre-fill forms; raw API response also returned for debugging

FR-APP-003: Bulk Import from Excel

FIELD	DETAIL
Priority	Medium

FIELD	DETAIL
Description	Admins shall import ITS numbers and applicant data in bulk via Excel file
Acceptance Criteria	Excel template downloadable; import returns per-row result (inserted/skipped/errors); bulk ITS API fetch available to hydrate imported records automatically

---

7.6 Counseling Forms Module

FR-CF-001: Multi-Section Counseling Form

FIELD	DETAIL
Priority	Critical
Description	The system shall provide an 8-section digital counseling form linked to each case, replacing paper-based counseling documentation
Acceptance Criteria	One counseling form per case, auto-created on case creation; sections: Personal Details, Family Details, Assessment, Financial Assistance, Economic Growth, Declaration, Attachments, Manzoori; each section independently editable and savable; completion tracked per section

Counseling Form Sections:

SECTION	KEY DATA CAPTURED
1. Personal Details	ITS, name, age, education, Jamiat/Jamaat, contact, email, residential address, present occupation, occupation address
2. Family Details	Family structure, wellbeing assessment (food/housing/education/health/deeni), income/expense breakdown (monthly & yearly), surplus/deficit, assets, liabilities, family members table (name, age, relation, education, occupation, annual income)
3. Assessment	Background assessment (education, work experience, family business, skills); proposed business details (present condition, sourcing, selling, major expenses, goods purchase, revenue, profit margin); counselor assessment (demand/supply, growth potential, competition strategy, support needed)
4. Financial Assistance	Assistance required, self-funding, rahen available, support needed (multi-select), QH repayment fields, timeline of assistance (purpose/Enayat/Qardan/months), action plan by timeline period, support mentors (ITS-fetched)
5. Economic Growth	6-year projections (last year + years 1–5) for: business assets, revenue/sales, expenses (raw material, labor, rent, overhead, repair), profit, fund blocked, Qardan repayment, other income, household expense, cash surplus
6. Declaration	Applicant confirmation, ITS, name, contact, date, signature (upload or draw); counselor confirmation, ITS, name, signature; TR committee ITS, name, signature
7. Attachments	Work place photo, quotation, product brochure, income tax return, financial statements, cancelled cheque, PAN card, Aadhar card, other documents
8. Manzoori (Approval)	Final approval signatures and sign-off documentation

FR-CF-002: Digital Signature Capture

FIELD	DETAIL
Priority	High
Description	The Declaration section shall support digital signature capture via file upload or on-screen drawing
Acceptance Criteria	Signature type: 'upload' (file path stored) or 'draw' (SVG/canvas data stored); applies to applicant, counselor, and TR committee signatures; custom SignaturePad component renders on-screen drawing capability

FR-CF-003: Counseling Form Section Permissions

FIELD	DETAIL
Priority	High
Description	Section-level read/write access on counseling forms shall be configurable per role via admin settings
Acceptance Criteria	Permissions stored in `counseling_form_stage_permissions` with `role_id`, `stage_key`, `can_read`, `can_update`; enforced at API middleware level; admin UI allows toggling per role per section

---

7.7 Cover Letter Module

FR-CL-001: Cover Letter Form

FIELD	DETAIL
Priority	High
Description	Authorized users shall complete a structured cover letter form with applicant financial summaries, recommendations, and approval signatures
Acceptance Criteria	Form fields: applicant and counselor identity (auto-fetched), current income/family financials, assets/liabilities, business details, requested Enayat/Qardan, recommended amounts, 5-year income projections, proposed upliftment plan, non-financial assistance, welfare/ZI/operations head signatures (upload or draw), approval dates; form supports partial save; form marked 'submitted' via dedicated submit action

FR-CL-002: PDF Cover Letter Generation

FIELD	DETAIL
Priority	High
Description	The system shall generate a formatted PDF cover letter from the cover letter form data
Acceptance Criteria	Generated via PDFKit / Puppeteer; stored on filesystem; downloadable via `GET /api/cover-letters/download/:id`; multiple cover letters can exist per case (re-generation supported); generation timestamp and generating user recorded

---

7.8 Payment Management Module

FR-PAY-001: Create Payment Schedule

FIELD	DETAIL
Priority	High
Description	Authorized users shall define payment disbursement schedules for approved cases
Acceptance Criteria	Payment type: 'qardan_hasana' or 'enayat'; fields: year_number (1–5), disbursement_year, disbursement_month, disbursement_amount, repayment_months (for QH), repayment_start_year, repayment_start_month; repayment installments auto-calculated and stored in `payment_schedule_repayments`; multiple yearly schedule rows per case

FR-PAY-002: Confirm Disbursement

FIELD	DETAIL
Priority	High
Description	Finance users shall confirm actual disbursement of scheduled payments
Acceptance Criteria	Records: disbursed_date (actual date), disbursed_by (user ID), disbursed_at (confirmation timestamp); `is_disbursed` flag set to true; optional repayment_months update at disbursement time

---

7.9 Fund Utilization Module

FR-FU-001: Quarterly Fund Utilization Reports

FIELD	DETAIL
Priority	High
Description	Authorized users shall create quarterly fund utilization reassessment reports per case, documenting how disbursed funds have been utilized
Acceptance Criteria	One report per quarter per year per case; quarter values: Q1–Q4; responses stored as JSON against dynamic form questions; photo documentation uploaded per report (shop/site, stock/equipment, applicant work, other); file attachments uploadable per question; PDF report generated via PDFKit on demand

FR-FU-002: Configurable Form Builder

FIELD	DETAIL
Priority	High
Description	Admins shall configure the fund utilization form structure by defining sections and questions without code changes
Acceptance Criteria	Admin can create/edit/delete/reorder sections; within each section, admin can create questions with field types: text, number, select, file upload, table; question properties: label, placeholder, is_required, is_auto_fill, auto_fill_key, min/max length, max file size/count, options (for select), table_config (JSON); conditional logic: parent_question_id + parent_trigger_values for show/hide

FR-FU-003: Auto-Fill Support

FIELD	DETAIL
Priority	Medium
Description	Questions with `is_auto_fill = true` shall be pre-populated from known case/applicant data
Acceptance Criteria	Auto-fill key (e.g., 'mauze') maps to applicant/case data fields; frontend pre-fills the field value; user can override the pre-filled value before saving

---

7.10 Welfare Checklist Module

FR-WC-001: Checklist Administration

FIELD	DETAIL
Priority	Medium
Description	Admins shall configure welfare checklist categories and items with drag-and-drop ordering
Acceptance Criteria	Categories: name, description, sort_order, is_active; items: category_id, form_section, checklist_detail, sort_order, is_compulsory; reorder endpoint updates sort_order for all moved items atomically; soft delete preserves data

FR-WC-002: Checklist Completion

FIELD	DETAIL
Priority	High
Description	Welfare reviewers shall complete welfare checklists per case, recording YES/NO/NA per item
Acceptance Criteria	Response options: Y, N (stored as enum); optional per-item comment; overall_remarks field for general notes; completion status endpoint returns total/filled/isComplete/completionPercentage; compulsory items must be addressed for checklist to be considered complete

---

7.11 Workflow & Approval Module

FR-WF-001: Configurable Workflow Stages

FIELD	DETAIL
Priority	Critical
Description	Admins shall configure approval workflow stages per case type with role/user assignments and SLA settings
Acceptance Criteria	Stage fields: stage_name, stage_key, sort_order, case_type_id, associated_statuses (JSON array), sla_value, sla_unit (hours/days/business_days/weeks/months), sla_warning_value, sla_warning_unit; per-stage role assignments with granular permissions (can_approve, can_reject, can_review, can_view, can_edit, can_delete, can_create_case, can_fill_case); per-stage user assignments with same permissions; drag-and-drop reordering; soft delete and restore

FR-WF-002: SLA Tracking

FIELD	DETAIL
Priority	High
Description	The system shall calculate and report SLA status for each case at its current workflow stage
Acceptance Criteria	SLA calculated from `current_stage_entered_at` against `sla_value` + `sla_unit`; statuses: on_time, at_risk (within warning threshold), breached; `sla_breached_at` timestamp recorded when breach occurs; SLA check triggerable via `POST /api/notifications/check-sla`; notifications created for at_risk and breached cases

FR-WF-003: Executive Approval Hierarchy

FIELD	DETAIL
Priority	High
Description	Cases requiring executive sanction shall progress through up to 5 sequential executive approval levels
Acceptance Criteria	Executive levels defined in `executive_levels` table with level_number and sort_order; cases track `current_executive_level`; approval by level N advances to level N+1; final level approval advances to next workflow stage (Manzoori); rework by any level sends case back to welfare department; executive comments support visibility toggle for DCM

7.12 Dashboard & Analytics Module

FR-DASH-001: Role-Specific Dashboards

FIELD	DETAIL
Priority	High
Description	The system shall provide role-tailored dashboard statistics, pipeline views, and recent activity feeds

ROLE	DASHBOARD CONTENT
Admin / Super Admin	Total cases by status, user stats (total/active/inactive by role), recent cases, system-wide pipeline
DCM / ZI	Assigned cases (total, pending assignment, in-counseling), recent assigned cases, pipeline for their scope
Counselor	Assigned cases (total, active counseling), recent counselor cases
Welfare Reviewer	Pending review count, approved count, rejected count, pending review queue
Executive	Pending approval at their level, approved count, rejected count, pending queue
Finance	Pending disbursement cases, disbursed count, pending disbursement queue

---

7.13 Notification System Module

FR-NOTIF-001: Automated Notifications

FIELD	DETAIL
Priority	High
Description	The system shall automatically create in-app notifications for key case events

EVENT	RECIPIENTS	TYPE
Case created / assigned	Assigned counselor	info
Case status changed	DCM, assigned counselor, relevant approver	info / success / warning
Case rejected / rework requested	DCM, assigned counselor	warning / error
SLA approaching (at_risk)	Assigned DCM, relevant approver	warning
SLA breached	Assigned DCM, relevant approver, admin	error
Executive approval received	DCM	success

FR-NOTIF-002: Notification Management

FIELD	DETAIL
Priority	Medium
Description	Users shall view, manage, and clear their notification history
Acceptance Criteria	Paginated notification list with case context; unread count badge in navigation; mark individual or all notifications as read; delete individual notifications; `GET /api/notifications/unread-count` for badge polling

7.14 Reporting Module

FR-RPT-001: PDF Report Generation

FIELD	DETAIL
Priority	High
Description	The system shall generate PDF reports for fund utilization quarterly reports and cover letters
Acceptance Criteria	Quarterly report PDF: generated via PDFKit + Puppeteer HTML template; filename: `Q{quarter}_{year}_Reassessment_Case-{case_number}.pdf` ; includes all form responses, photos, and signatures; Cover letter PDF: generated via PDFKit; includes all cover letter form data and signatures

FR-RPT-002: Excel Import/Export

FIELD	DETAIL
Priority	Medium
Description	The system shall support Excel-based bulk import and export for users, applicants, and Jamiat/Jamaat master data
Acceptance Criteria	Downloadable templates for each entity; import returns row-level result summary; export generates `.xlsx` files via the `xlsx` library; import validation reports errors per row without halting the entire batch

8. Non-Functional Requirements

8.1 Performance

ID	REQUIREMENT	TARGET
NFR-PERF-001	API response time (standard endpoints)	< 500 ms at p95
NFR-PERF-002	Case list page load (100 records)	< 2 seconds
NFR-PERF-003	File upload processing (10 MB)	< 10 seconds
NFR-PERF-004	PDF report generation (quarterly)	< 8 seconds
NFR-PERF-005	ITS external API lookup	< 3 seconds (network-dependent)
NFR-PERF-006	Concurrent users supported	50+ simultaneous sessions
NFR-PERF-007	Database connection pool	10 persistent connections (configurable)

8.2 Reliability

ID	REQUIREMENT	TARGET
NFR-REL-001	System availability	99.5% uptime excluding planned maintenance
NFR-REL-002	Data integrity	All DB writes use parameterized queries; InnoDB transactions enforced
NFR-REL-003	ITS API degradation	Graceful fallback to manual entry if ITS endpoint is unreachable
NFR-REL-004	Error handling	All API errors return structured JSON; no raw stack traces exposed to client
NFR-REL-005	Audit trail	Workflow logs, income change audit, and status history preserved indefinitely

8.3 Security

ID	REQUIREMENT	TARGET
NFR-SEC-001	Authentication	JWT tokens with 24-hour expiry; token verified on every request
NFR-SEC-002	Password storage	bcryptjs hashing; plain-text passwords never stored
NFR-SEC-003	SQL injection prevention	100% parameterized queries via mysql2
NFR-SEC-004	HTTP security headers	Helmet.js middleware on all responses
NFR-SEC-005	CORS	Restricted to configured frontend origin only
NFR-SEC-006	Rate limiting	Login endpoint rate-limited to prevent brute-force
NFR-SEC-007	File upload validation	MIME type and file-size validation via Multer; max 10 MB per file
NFR-SEC-008	Organizational scope isolation	Jamiat/Jamaat boundaries enforced at SQL query level for all non-admin roles
NFR-SEC-009	Input validation	Joi schema validation on all request bodies before DB access
NFR-SEC-010	Token status check	User `is_active` flag verified on every authenticated request

8.4 Usability

ID	REQUIREMENT	TARGET
NFR-USE-001	Browser compatibility	Chrome, Firefox, Safari, Edge (latest 2 major versions)
NFR-USE-002	Responsive design	Tailwind CSS responsive grid; functional on 768px+ viewport widths
NFR-USE-003	Form feedback	Real-time validation errors via React Hook Form; toast notifications for all actions
NFR-USE-004	Navigation	Role-specific sidebar menus; breadcrumb navigation on detail pages

ID	REQUIREMENT	TARGET
NFR-USE-005	Loading states	Spinner indicators for all async data loads; optimistic updates via React Query

8.5 Scalability

ID	REQUIREMENT	TARGET
NFR-SCA-001	Database indexing	Indexed on all FK columns, `its_number`, `status`, `case_number`
NFR-SCA-002	Pagination	All list endpoints paginated with configurable `limit` (10–500)
NFR-SCA-003	File storage	Local filesystem with structured paths; S3 migration path available
NFR-SCA-004	Process management	PM2 cluster mode or systemd for multi-core utilization

8.6 Maintainability

ID	REQUIREMENT	TARGET
NFR-MNT-001	Code organization	25 route files; services/utils separation; React component library
NFR-MNT-002	Database migrations	Individual SQL migration files in `database/migrations/`; consolidated production dump maintained
NFR-MNT-003	Environment configuration	All secrets in `.env`; no hardcoded credentials in source
NFR-MNT-004	Audit columns	`created_by`, `updated_by`, `created_at`, `updated_at` on all core tables
NFR-MNT-005	Documentation	README, API schema reference, DB schema, and this BRD maintained in sync

9. System Architecture

9.1 Component Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CLIENT TIER (React 18 SPA)                    │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐   │
│  │ Auth Context│  │ React Query  │  │  React Router v6          │ │
│  │ (JWT store) │  │ (Server State│  │  (Protected Routes)       │ │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘   │
│                                                                      │
│  Pages (31):  Login · Dashboard · Cases · CaseDetails ·              │
│               CaseIdentification · Applicants · Users ·              │
│               RoleManagement · JamiatMaster · WorkflowStages ·       │
│               WelfareChecklist · FundUtilization · Reports ·         │
│               PaymentSchedule · Notifications · Profile              │
│                                                                      │
│  Components: CounselingForm · CoverLetterForm · PaymentSchedule      │
│              FundUtilization · WelfareChecklistForm · SignaturePad |
│              FileUpload · CaseSLAStatus · WorkflowComments           │
│  UI Library (16): Button · Card · Modal · Table · Alert · Badge · |
│                   Input · Select · SearchableSelect · MultiSelect  |
│                   Pagination · Toast · Tabs · Spinner · Progress   |
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │ HTTPS / JSON (Axios)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                   API GATEWAY TIER (Node.js + Express)               │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │ Middleware Stack                                          |       │
│  │ Helmet (security headers) → CORS → Rate Limiter         |       |
│  │ → JWT Auth → RBAC Permissions → Joi Validation          |       |
│  └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│  Route Modules (25):                                                 │
│  auth · users · cases · applicants · caseIdentifications             │
│  counselingForms · coverLetters · coverLetterForms                   │
│  fundUtilization · fundUtilizationMaster · quarterlyReportPdf        │
│  dashboard · notifications · attachments · roles · permissions       │
│  workflowStages · welfareChecklist · jamiat · jamaat                 │
│  caseTypes · relations · educationLevels · occupations               │
│  executiveLevels · businessAssets                                    │
│                                                                      │
│  Services: emailService · notificationService                        │
│  Utils:    slaCalculator · permissionUtils · roleUtils               │
│            activeCasePerIts · quarterlyReportPdfData/Kit             │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │ mysql2 (parameterized queries)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              DATA TIER                               │
│                                                                      │
│  ┌──────────────────────────┐  ┌──────────────────────────────┐     │
│  │ MySQL 8.0 (InnoDB)       │  │ Local Filesystem             │     │
│  │ DB: baaseteen_cms        │  │ /uploads/ (Multer)           │     │
│  │ 50+ tables               │  │ PDFs · images · docs         │     │
│  │ Connection pool: 10      │  │ Max 10 MB per file           │     │
│  └──────────────────────────┘  └──────────────────────────────┘     │
│                                                                      │
│  ┌──────────────────────────┐                                        │
│  │ External ITS API         │                                        │
│  │ (applicant data fetch)   │                                        │
│  │ GET /fetch-from-api/:its |                                        │
│  └──────────────────────────┘                                        │
└──────────────────────────────────────────────────────────────────────┘
```

9.2 Request Lifecycle

```
1. React SPA makes Axios HTTP request with Bearer token
       |
2. Express receives request
   → Helmet adds security headers
   → CORS validates origin
   → Rate limiter checks request frequency
       |
3. JWT Auth Middleware:
   - Decodes and verifies JWT token
   - Checks user.is_active in DB
   - Attaches user object to req.user
   - Rejects with 401 if invalid / expired
       |
4. RBAC Permission Middleware:
   - Checks req.user.role against required resource:action
   - For scoped roles: applies Jamiat/Jamaat filters to query
   - Rejects with 403 if insufficient permission
       |
5. Joi Validation:
   - Validates request body / query params against schema
   - Returns 400 with field-level errors on failure
       |
6. Route Handler (Business Logic):
   - Executes parameterized MySQL queries via connection pool
   - Calls services (emailService, notificationService, slaCalculator)
   - Invokes PDF/Excel generators as needed
   - Handles file I/O via Multer
       |
7. Response:
   - Returns JSON payload with appropriate HTTP status
   - React Query caches and updates UI state
   - Toast notification displayed to user
```

9.3 Deployment Architecture

```
┌──────────────────────────────────────────────────────────┐
│                     Production Server                    │
│                                                          │
│  ┌──────────────────────────────────────────────────┐   │
│  │  Nginx Reverse Proxy (443 / SSL)                 │   │
│  │  location /     → React build (static files)     │   │
│  │  location /api  → localhost:5000 (Express)       │   │
│  │  location /uploads → localhost:5000/uploads      │   │
│  │  client_max_body_size 10M                        │   │
│  └──────────────────────────────────────────────────┘   │
│                          |                               │
│  ┌──────────────────────┴──────────────────────────┐   │
│  │  Node.js Express (port 5000)                     │   │
│  │  Managed by PM2 or systemd                       │   │
│  └──────────────────────────────────────────────────┘   │
│                          |                               │
│  ┌──────────────────────┴──────────────────────────┐   │
│  │  MySQL 8.0 (localhost)                           │   │
│  │  DB: baaseteen_cms                               │   │
│  └──────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

---

10. Database Design

10.1 Entity-Relationship Overview

```
users ──────────────────┬─── user_roles ─── roles ─── role_permissions
                        │
                        ├─── cases ──────────┬── applicants ── jamiat / jamaat
                        │         │          │
                        │         │          ├── case_types
                        │         │          ├── workflow_stages ── workflow_stage_roles
                        │         │          ├── statuses
                        │         │          └── status_history
                        │         │
                        │         ├── counseling_forms
                        │         │       ├── personal_details
                        │         │       ├── family_details ── family_members
                        │         │       ├── assessment ── assessment_products_services
                        │         │       ├── financial_assistance
                        │         │       │       ├── financial_assistance_timeline
                        │         │       │       ├── financial_assistance_timeline_assistance
                        │         │       │       ├── financial_assistance_action_plan
                        │         │       │       ├── financial_assistance_mentors
                        │         │       │       └── financial_assistance_qh_repayment_schedule
                        │         │       ├── economic_growth ── economic_growth_projections
                        │         │       ├── declaration
                        │         │       └── attachments
                        │         │
                        │         ├── cover_letter_forms / cover_letters
                        │         ├── case_identifications ── case_identification_income_logs
                        │         ├── case_comments / workflow_comments / case_attachments
                        │         ├── case_closures ── case_closure_documents
                        │         ├── payment_schedules ── payment_schedule_repayments
                        │         ├── fund_utilization_reports
                        │         │       ├── fund_utilization_photos
                        │         │       └── fund_utilization_files
                        │         ├── welfare_checklist_responses
                        │         ├── business_assets
                        │         └── notifications
                        │
                        └── fund_utilization_sections
                                └── fund_utilization_questions (self-ref)
```

10.2 Core Tables

TABLE	PURPOSE	PRIMARY KEY	KEY COLUMNS
`users`	All user accounts and credentials	`id`	`username`, `role`, `executive_level`, `is_active`
`roles`	Role definitions with permission JSON	`id`	`name`, `display_name`, `is_system_role`
`user_roles`	User-to-role assignments with scope	`id`	`user_id`, `role_id`, `jamiat_ids`, `jamaat_ids`
`role_permissions`	Granular resource:action permissions per role	`id`	`role_id`, `resource`, `action`
`cases`	Main case records and workflow state	`id`	`case_number`, `status`, `current_workflow_stage_id`, `is_duplicate`
`applicants`	Welfare applicant profiles	`id`	`its_number`, `full_name`, `jamiat_id`, `jamaat_id`
`case_types`	Case classification types (Baaseteen, SHND)	`id`	`name`, `is_active`
`statuses`	Case status definitions with colors	`id`	`name`, `color`, `is_final`
`jamiat`	Parent organizations	`id`	`jamiat_id`, `name`, `is_active`

TABLE	PURPOSE	PRIMARY KEY	KEY COLUMNS
`jamaat`	Sub-organizations linked to Jamiat	`id`	`jamaat_id`, `jamiat_id`, `name`

10.3 Workflow & Audit Tables

TABLE	PURPOSE	PRIMARY KEY
`workflow_stages`	Configurable approval stages per case type with SLA settings	`id`
`workflow_stage_roles`	Role permissions per workflow stage (approve/reject/view flags)	`id`
`workflow_stage_users`	User-level overrides for workflow stage permissions	`id`
`workflow_comments`	Stage-specific comments and feedback during workflow transitions	`id`
`status_history`	Immutable audit log of every case status change	`id`
`case_comments`	General and typed comments attached to cases	`id`
`notifications`	In-app notification records per user per case	`id`
`case_identification_income_logs`	Audit trail for income field edits on case identifications	`id`

10.4 Counseling Form Tables

TABLE	PURPOSE
`counseling_forms`	Master form record linking all 8 section tables to a case; tracks completion
`personal_details`	Section 1: ITS, name, education, contact, address, occupation
`family_details`	Section 2: Wellbeing, income/expense breakdown, assets/liabilities
`family_members`	Individual family members linked to `family_details`
`assessment`	Section 3: Background, proposed business details, counselor assessment
`assessment_products_services`	Products/services table within assessment
`financial_assistance`	Section 4: Assistance required, self-funding, Rahen, QH fields
`financial_assistance_timeline`	Purpose/cost/Enayat/Qardan/months per funding tranche
`financial_assistance_timeline_assistance`	Phased assistance plan (immediate / after 1st yr / etc.)
`financial_assistance_action_plan`	Action steps per timeline period
`financial_assistance_mentors`	Support mentor details (ITS, name, contact, photo)
`financial_assistance_qh_repayment_schedule`	Qardan Hasana repayment schedule per year
`economic_growth`	Section 5: 6-year projected P&L (last year + years 1–5)
`economic_growth_projections`	Custom projection rows per economic growth record
`declaration`	Section 6: Digital signatures for applicant, counselor, TR committee
`attachments`	Section 7: Checklist flags for required document uploads
`counseling_form_stage_permissions`	Role-level read/update permissions per counseling form stage

10.5 Financial & Fund Tables

TABLE	PURPOSE
`payment_schedules`	Annual disbursement plans (Qardan Hasana or Enayat) with disbursement confirmation
`payment_schedule_repayments`	Monthly repayment rows per payment schedule

TABLE	PURPOSE
`cover_letter_forms`	Cover letter form data including all signature fields and financial summaries
`cover_letters`	Generated cover letter PDF file references
`fund_utilization_reports`	Quarterly fund utilization report per case (quarter, year, JSON responses)
`fund_utilization_photos`	Photo documentation per fund utilization report
`fund_utilization_files`	File uploads per question per fund utilization report
`fund_utilization_sections`	Admin-configurable form sections for fund utilization builder
`fund_utilization_questions`	Dynamic form questions with type, validation, auto-fill, and conditional logic
`business_assets`	6-year business asset projections (8 asset categories × 6 time periods)

10.6 Welfare Checklist & Supporting Tables

TABLE	PURPOSE
`welfare_checklist_categories`	Grouping categories for checklist items (admin-managed)
`welfare_checklist_items`	Individual checklist items with compulsory flag
`welfare_checklist_responses`	Per-case responses (Y/N/NA) with comments
`case_identifications`	Pre-assessment eligibility screening records with income and family data
`case_attachments`	File upload metadata for case documents (stage-tagged)
`case_closures`	Case closure records with reason and document references
`case_closure_documents`	Individual closure document files
`executive_levels`	Executive approval hierarchy levels with ordering
`relations`, `education_levels`, `occupations`	Master data lookup tables
`system_settings`	Key-value configuration store

10.7 Key Design Decisions

1. **InnoDB engine** — All tables use InnoDB for foreign key constraint enforcement and ACID transactions
2. **Organizational scoping at SQL level** — Jamiat/Jamaat filters injected into queries based on `user_roles.jamiat_ids` / `jamaat_ids` columns
3. **One-active-case-per-ITS enforcement** — Utility `activeCasePerIts.js` prevents duplicate active cases for the same applicant
4. **JSON columns** — `workflow_history`, `associated_statuses`, `responses_json` stored as JSON/longtext for flexibility
5. **Soft delete pattern** — Workflow stages and checklist items use `is_active` flag instead of hard deletion
6. **Self-referencing FK** — `workflow_stages.next_stage_id` and `fund_utilization_questions.parent_question_id` implement linked-list stage chains and conditional form logic
7. **Base64 photo storage** — Applicant and user photos stored as Base64 longtext in DB (avoids filesystem dependency for profile images)
8. **Decimal precision** — All financial fields use `DECIMAL(15,2)` for currency accuracy

---

11. User Interface Requirements

11.1 Global Layout

- **Left Sidebar**: Role-specific navigation links with icons; collapsible on mobile
- **Top Header**: Organization logo, current user name/role, notification bell with unread badge, profile menu
- **Content Area**: Full-width responsive Tailwind grid; page-specific content with breadcrumb trail
- **Toast System**: Bottom-right toast notifications for all async action results
- **Modal System**: Centered overlay modals for confirmations, form dialogs, and detail views

11.2 Route Protection Strategy

All routes in `App.js` use one of the following guard patterns:

GUARD	USAGE
`requireAdminAccess`	Admin or Super Admin only pages (User management, Role management)
`requiredPermission(resource, action)`	Single permission check (e.g., `cases:create`)
`requiredAnyOfPermissions([...])`	Any matching permission from a list (mixed-role pages)
`requireRoles([...])`	Explicit role whitelist (e.g., Executive approval page)
`requireCounselingFormAccess`	Counseling form stages access via role stage permissions

11.3 Key UI Pages

PAGE	ROUTE	ACCESS	KEY ELEMENTS
Login	`/login`	Public	Centered card, username/password, JWT token stored in AuthContext
Dashboard	`/dashboard`	All	Role-specific stat cards, case pipeline chart, recent activity feed
Cases	`/cases`	cases:read	Paginated table with 8+ filter options, SLA status badges, quick actions
Case Details	`/cases/:id`	Case access	Tabbed layout: Overview, Counseling Form, Cover Letter, Payments, Fund Utilization, Checklist, Attachments
Case Identification	`/case-identification`	case_identification:read	ITS auto-fetch, income assessment form, eligible/ineligible review actions
Applicants	`/applicants`	applicants:read	Applicant list, ITS lookup, bulk import/fetch from ITS API
Workflow Stages	`/workflow-stages`	master:read	Drag-and-drop reorder, role/user assignment, SLA configuration per stage
Fund Utilization Admin	`/fund-utilization-master`	Admin	Dynamic form builder — add/reorder sections and questions, configure field types and validation
Role Management	`/roles`	super_admin	Role CRUD, granular permission checkboxes per resource:action, counseling form stage permissions
Reports	`/reports`	reports:view	PDF download links for quarterly reports and cover letters

11.4 Component Library

All UI components reside in `frontend/src/components/ui/` and are used system-wide:

COMPONENT	PURPOSE
`Button`	Primary / secondary / danger / ghost variants with loading state
`Card`	Content container with optional header and footer slots

COMPONENT	PURPOSE
`Modal`	Accessible overlay dialog with close-on-backdrop and escape key
`Table`	Sortable, paginated data table with column slot API
`Alert / Toast`	Contextual messages (info / success / warning / error)
`Badge`	Status pill with color mapping per case status
`Input / Select`	Controlled form inputs with error state display
`SearchableSelect`	Dropdown with client-side search (used for ITS, Jamiat, Counselor selection)
`MultiSelect`	Multi-value select for role/Jamiat/Jamaat assignment
`Pagination`	Page controls wired to React Query `page` state
`Tabs`	Tab bar for Case Details sectional navigation
`Spinner`	Loading indicator for async data fetches
`WorkflowProgress`	Visual step indicator showing current workflow stage and history

11.5 Design Specifications

- **Framework**: Tailwind CSS v3.4 with custom theme config
- **State management**: React Query (server state) + React Context (auth) + useState (UI state)
- **Form handling**: React Hook Form v7 with Zod/custom validation
- **Charts**: Recharts v2.8 for dashboard pipeline and statistics
- **Drag & Drop**: @dnd-kit for workflow stage and checklist reordering
- **Signatures**: Custom SignaturePad component supporting draw and file-upload modes
- **HTTP client**: Axios v1.6 with interceptors for JWT injection and error handling

---

12. Data Flow Diagrams

12.1 Case Lifecycle Flow

```
Field Worker / DCM              System                      Approver / Executive
       |                          |                                |
       ├── Create Case ID ────────►|                                |
       |   (ITS lookup → auto-fill)|                               |
       |                          ├── ITS API fetch                |
       |                          ├── Duplicate ITS check          |
       |◄── Case ID record saved ──|                               |
       |                          |                                |
       |◄──────────── Reviewer marks Eligible / Ineligible ─────────►|
       |                          ├── If Eligible: Auto-create Case|
       |                          ├── Assign DCM + Counselor       |
       |◄── Case created (Draft) ──|                               |
       |                          |                                |
       ├── Fill Counseling Form ──►| (8 sections, section by section)
       |                          ├── Save per section             |
       |◄── Form progress tracked ─|                               |
       |                          |                                |
       ├── Submit for Welfare ────►|                               |
       |                          ├── Notify Welfare Reviewer ─────►|
       |                          |◄── Welfare Approve / Reject ────|
       |                          ├── Notify DCM ◄────────────────|
       |                          ├── Advance to ZI Approval       |
       |                          |◄── ZI Approve / Reject ────────|
       |                          ├── Advance to KG / Ops Lead ────|
       |                          ├── Cover Letter Generation       |
       |                          ├── Executive Approval (1-5 lvls)|
       |                          ├── Manzoori (Sanction)          |
       |                          ├── Financial Disbursement       |
       |                          ├── Payment Schedule created     |
       |                          ├── Quarterly Fund Utilization   |
       |                          └── Case Closure
```

12.2 SLA Calculation Flow

```
Case moves to new workflow stage
       |
       ├── Record current_stage_entered_at = NOW()
       |
SLA Check triggered (manual POST /check-sla or event-based)
       |
       ├── For each case in non-terminal status:
       |       elapsed = NOW() - current_stage_entered_at
       |       stage.sla_value / sla_unit → convert to hours
       |       stage.sla_warning_value / sla_warning_unit → convert
       |
       ├── If elapsed ≥ sla_value:
       |       status = "breached"
       |       record sla_breached_at
       |       notify: DCM + relevant approver + admin (error notification)
       |
       ├── If elapsed ≥ sla_warning_value (but < sla_value):
       |       status = "at_risk"
       |       notify: DCM + relevant approver (warning notification)
       |
       └── Else: status = "on_time"
           (returned as sla_status field on case list API)
```

12.3 ITS Auto-Fetch Flow

```
User enters ITS number in Case Identification / Applicant form
        │
        ├── Frontend: GET /api/applicants/fetch-from-api/:itsNumber
        │
        ├── Backend checks local DB first (applicants table)
        │       If found → return with source: "database"
        │
        ├── If not in DB → call external ITS API endpoint
        │       Parse response → map to applicant schema
        │       Return with source: "api"
        │
        ├── Frontend auto-populates form fields:
        │       full_name, age, gender, phone, email, photo,
        │       jamiat_name, jamaat_name, address
        │
        └── User reviews, edits if needed, then saves record
```

---

13. Module Dependencies

```
┌──────────────────────────────────────────────────────────────┐
│              Authentication & RBAC Module                    │
│      JWT + user_roles + role_permissions                     │
│      (Required by ALL modules — every route is guarded)      │
└──────────────────────────────────────────────────────────────┘
                            │
                            │
┌───────────────────────────┼──────────────────────────────────┐
│              User Management & Role Management                 │
│      (Required for counselor/DCM/approver assignment)          │
└──────┬────────────────────┼──────────────────────────────────┘
       │                    │
┌──────┴──────────┐    ┌────┴──────────────────────────────┐
│  Master Data    │    │         Case Identification Module      │
│  Jamiat/Jamaat  │    │  (Eligibility screening — entry point)  │
│  CaseTypes      │    └──────────────────┬──────────────────────┘
│  Relations      │                       │ (eligible → auto-create)
│  EducationLvls  │                       ▼
│  Occupations    │    ┌──────────────────────────────────────┐
│  ExecutiveLvls  │    │           Case Management Module        │
│  WorkflowStages │    │  (core — all other modules depend on it)│
└─────────────────┘    └──┬──┬──┬──┬──┬──┬──┬──┬──┬──────────────┘
                          │  │  │  │  │  │  │  │  │
       ┌──────────────────┘  │  │  │  │  │  │  │  └──────────────┐
       ▼                     │  │  │  │  │  │  │                 ▼
Counseling Forms        Payment  │  │  │  │  │  Welfare        Notifications
(8-section form)       Schedule  │  │  │  │  │  Checklist      (cross-cutting)
                                 │  │  │  │  │
                          Cover  │  │  │  Fund Utilization
                          Letter │  │  │  (quarterly reports)
                                 │  │  │
                        Attach-  │  Dashboard & Analytics
                        ments    │  (reads from all modules)
                                 │
                            Audit Trail
                            (status_history, workflow_comments,
                             income_logs — immutable writes)

Cross-cutting: SLA Calculator (reads workflow_stages, writes sla_status)
Cross-cutting: Notification Service (triggered by Case, Workflow, SLA events)
Cross-cutting: Email Service (optional SMTP layer over Notification Service)
```

---

14. Integration Points

14.1 Internal Integrations

INTEGRATION	DESCRIPTION	MECHANISM
Auth ↔ All Modules	Every API route validates JWT and checks RBAC before executing	Express middleware chain
Case Identification ↔ Case Management	Marking a case identification as eligible auto-creates a case record	Direct DB insert in route handler
Counseling Form ↔ Cover Letter Form	Cover letter form pre-populates from counseling form sections (income, assets, projections)	API aggregation in `GET /cover-letter-forms/case/:id`
Workflow Stages ↔ Cases	Each workflow action advances `current_workflow_stage_id` and appends to `workflow_history`	DB update + status history insert
SLA Calculator ↔ Workflow Stages	SLA values from `workflow_stages` applied against `current_stage_entered_at` on each case	`slaCalculator.js` utility
Notification Service ↔ Case Events	In-app notifications created for status changes, assignments, SLA breaches	`notificationService.js` called from route handlers
Fund Utilization Master ↔ Reports	Dynamic form sections/questions drive the quarterly report form render	API join on section/question IDs
Attachment Module ↔ Fund Utilization	Photo uploads and file question answers stored via fund utilization photos/files tables	Multer upload + DB insert

14.2 External Integrations

INTEGRATION	TECHNOLOGY	PURPOSE	FALLBACK
ITS Institutional API	External REST API	Auto-fetch applicant demographics and photo by ITS number	Manual data entry if API unreachable
SMTP Email	Nodemailer v6.9	Optional email alerts for case status changes	In-app notifications still sent; email silently skipped
PDFKit	Node.js library v0.14	Cover letter PDF generation	N/A — required for cover letters
Puppeteer	Headless Chrome v24	Quarterly fund utilization report PDF (HTML template → PDF)	N/A — required for quarterly PDFs
xlsx library	Node.js library v0.18	Excel template download and bulk import for users, applicants, Jamiat	CSV format acceptable as alternative
Multer	Node.js middleware v1.4	Multipart file upload handling (case attachments, closure docs, photos)	N/A — required for file uploads

15. Deployment Architecture

15.1 Production Environment

COMPONENT	SPECIFICATION
OS	Linux (Ubuntu recommended)
Web Server / Proxy	Nginx with SSL termination (Let's Encrypt or commercial cert)
Backend Runtime	Node.js v16+ managed by PM2 (cluster mode) or systemd
Frontend Delivery	React production build served as static files via Nginx `root` directive with `try_files` SPA fallback
Database	MySQL 8.0, database name: `baaseteen_cms`
File Storage	Local filesystem at `./uploads/` ; proxied through Nginx `location /uploads`
Backend Port	5000 (internal only; Nginx proxies `/api` and `/uploads`)
SSL	HTTPS required in production; HTTP → HTTPS redirect enforced via Nginx
Max upload size	10 MB (`client_max_body_size 10M` in Nginx; `MAX_FILE_SIZE=10485760` in .env)

15.2 Deployment Steps

```
# 1. Build React frontend
cd frontend && npm run build

# 2. Configure production .env
#    NODE_ENV=production, strong JWT_SECRET, production DB credentials

# 3. Import database schema
mysql -u root -p -e "CREATE DATABASE baaseteen_cms;"
mysql -u root -p baaseteen_cms < database/baaseteen_production_database.sql

# 4. Start backend with PM2
cd backend
pm2 start index.js --name facts-backend
pm2 save && pm2 startup

# 5. Configure Nginx
#    Serve /frontend/build as root
#    Proxy /api and /uploads to localhost:5000

# 6. Post-deployment checklist
#    ✓ Default admin password changed (admin / password → new secure password)
#    ✓ JWT_SECRET set to random 32+ character string
#    ✓ uploads/ directory has write permissions for node process
#    ✓ Database backups scheduled (daily recommended)
#    ✓ Firewall: only ports 80, 443, 22 open
```

15.3 Local Development Environment

COMPONENT	SPECIFICATION
Backend	`npm run backend` → nodemon on port 5000
Frontend	`npm run frontend` → React dev server on port 3000
Combined	`npm run dev` → concurrently starts both
Database	Local MySQL 8.0; `DB_HOST=localhost` in `.env`
API URL	`REACT_APP_API_URL=http://localhost:5000`
Default login	Username: `admin` / Password: `password` (change immediately)

16. Security Requirements

16.1 Authentication Security

REQUIREMENT	IMPLEMENTATION
Token-based auth	JWT signed with `JWT_SECRET` (min 32 chars); expires in 24 hours
Password hashing	bcryptjs — no plain-text passwords stored or logged
Active status check	`users.is_active` verified on every authenticated request; deactivated users immediately lose access
Login rate limiting	`express-rate-limit` on `POST /api/auth/login` to prevent brute-force
Default credential warning	Admin must change default password (`admin` / `password`) on first login

16.2 API & Data Security

REQUIREMENT	IMPLEMENTATION
SQL injection prevention	All DB queries use `mysql2` parameterized statements; no string concatenation in SQL
Input validation	Joi schemas validate all request bodies before any DB access
HTTP security headers	Helmet.js sets Content-Security-Policy, X-Frame-Options, HSTS, etc.
CORS restriction	Only `FRONTEND_URL` env var origin accepted; credentials: true
Scope isolation	Jamiat/Jamaat IDs from `user_roles` injected as SQL `WHERE IN` clauses for all non-admin queries
RBAC enforcement	Every route uses `permissionMiddleware` checking `resource:action` against user's role permissions

16.3 File Upload Security

REQUIREMENT	IMPLEMENTATION
MIME type validation	Multer `fileFilter` validates MIME types; extension spoofing rejected
File size limit	10 MB per file via Multer `limits.fileSize` and Nginx `client_max_body_size`
Upload count limits	Max 10 files per multi-upload request; max 10 closure documents per case
Secure file paths	Uploaded files stored with random UUID-based names; original filenames sanitized and stored in DB only
Download access control	All file download endpoints require authenticated JWT; case attachment downloads verify case access

16.4 Infrastructure Security

REQUIREMENT	IMPLEMENTATION
HTTPS enforcement	Nginx configured for SSL with HTTP → HTTPS redirect
Firewall	Only ports 22 (SSH), 80 (HTTP redirect), 443 (HTTPS) open externally
Environment secrets	All credentials in `.env` file; `.gitignore` excludes `.env` from version control
Database access	MySQL bound to localhost only; no external DB port exposure
Error masking	Production errors logged server-side; generic message returned to client

17. Performance Requirements

17.1 Response Time Targets

OPERATION	TARGET	NOTES
API response (list endpoints)	< 500 ms	Paginated, indexed queries
Case detail fetch (full)	< 800 ms	Joins across 10+ tables
Dashboard overview	< 1 second	Role-filtered aggregate counts
ITS external API fetch	< 3 seconds	Network-dependent; graceful timeout
File upload (10 MB)	< 10 seconds	Multer streaming to filesystem
Cover letter PDF generation	< 5 seconds	PDFKit
Quarterly report PDF	< 8 seconds	Puppeteer HTML → PDF render
Excel bulk import (100 rows)	< 5 seconds	Row-level validation + DB inserts
SLA check (all cases)	< 3 seconds	Batch calculation on active cases

17.2 Optimization Strategies

STRATEGY	IMPLEMENTATION
Database indexing	Indexes on all FK columns, `its_number`, `case_number`, `status`, `current_workflow_stage_id`
Connection pooling	mysql2 connection pool with 10 persistent connections
Pagination on all lists	All list APIs support configurable `limit` (10–500); defaults to 10–20
React Query caching	Server responses cached client-side; stale-while-revalidate pattern reduces redundant API calls
Selective column queries	List endpoints select only required columns; detail endpoints fetch full row
Base64 photo storage	Profile photos stored in DB (no filesystem round-trip for photos); large images compressed before storage
Multer streaming	File uploads streamed directly to disk; not buffered in memory

18. Assumptions and Constraints

18.1 Assumptions

ID	ASSUMPTION
A-1	All users have modern web browsers (Chrome, Firefox, Safari, Edge) and stable internet access
A-2	Each welfare applicant has a unique ITS number registered in the external ITS institutional system
A-3	The external ITS API is maintained and accessible by the hosting organization; credentials are provided separately
A-4	Organizational hierarchy (Jamiat → Jamaat) is pre-configured in the system before case operations begin
A-5	Workflow stages and case types are configured by a Super Admin prior to case creation
A-6	Concurrent active user count will not exceed 50 for the initial deployment
A-7	File storage growth will remain within server disk capacity; S3 migration can be adopted if needed
A-8	Financial figures in the system are informational; actual payment processing occurs through external banking channels
A-9	Email notifications are optional; in-app notifications remain the primary alerting channel
A-10	Administrators are technically capable of system configuration including role/permission and workflow setup

18.2 Constraints

ID	CONSTRAINT
C-1	Technology stack is fixed to Node.js + Express (backend) and React (frontend); framework migration is out of scope
C-2	File storage is local filesystem; no cloud object storage (S3/GCS) integration in current version
C-3	Puppeteer requires Chromium on the server; headless browser dependencies must be installed separately
C-4	ITS external API availability is outside the control of the FACTS development team; no guaranteed uptime SLA
C-5	No integrated payment gateway; disbursement amounts are recorded in FACTS but actual transfers are external
C-6	Single-server deployment (no horizontal scaling / load balancing) in current architecture
C-7	Real-time features use polling (unread notification count) rather than WebSockets
C-8	Timezone configuration is set per deployment environment; all timestamps stored in UTC by MySQL

19. Risks and Mitigation

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION
R-1	ITS external API unavailability	Medium	Medium	Graceful fallback to manual data entry; all ITS-fetched fields are editable
R-2	Database data loss / corruption	Low	Critical	Daily automated database backups; InnoDB transactions; foreign key constraints
R-3	Unauthorized access / privilege escalation	Low	High	JWT verification on every request; RBAC on every route; `is_active` check prevents deactivated user access
R-4	Duplicate active cases for same applicant	Medium	High	`activeCasePerIts.js` utility enforces one active case per ITS; `is_duplicate` flag surfaced in UI
R-5	SLA breach notification failures	Medium	Medium	Manual SLA check trigger available; SLA status visible on case list even without notifications
R-6	File storage disk exhaustion	Medium	Medium	10 MB file size limit; max 10 files per upload; monitor disk usage; plan S3 migration proactively
R-7	Puppeteer / Chromium PDF generation failure	Low	Medium	Error handling returns descriptive failure message; cover letters can be re-generated on demand
R-8	Server downtime during approval workflows	Low	High	PM2 auto-restart on process crash; systemd fallback; workflow state persisted in DB at each stage transition
R-9	Performance degradation at scale	Medium	Medium	All list endpoints paginated; DB fully indexed on FK and search columns; React Query reduces redundant requests
R-10	Misconfigured RBAC locking out users	Low	High	Super Admin role is protected system role; Super Admin always retains full access to correct misconfigured roles

20. Acceptance Criteria

20.1 Module-Level Acceptance

MODULE	CRITERIA	STATUS
Authentication	JWT login/logout; role-based navigation; deactivated user blocked immediately	✓
User Management	CRUD with multi-role assignment; Jamiat/Jamaat scope per role; bulk import; Excel template	✓
Role Management	Create/edit roles with granular resource:action permissions; counseling form stage permissions	✓
Case Identification	ITS auto-fetch; duplicate detection; eligible → auto case creation; income audit log; revert to pending	✓
Case Management	Case number auto-generated; multi-stage workflow; approve/reject/rework; SLA tracking; case closure with docs	✓
Applicant Management	ITS lookup from DB and API; bulk import; bulk ITS fetch; Excel template; one-active-case enforcement	✓
Counseling Forms	All 8 sections save independently; digital signature capture; form completion tracking; role-stage permissions	✓
Cover Letters	Form pre-populates from counseling form; PDF generated via PDFKit; download available	✓
Payment Management	Qardan Hasana and Enayat schedules; year/month/amount tracking; disbursement confirmation with date	✓
Fund Utilization	Dynamic form builder; quarterly reports; photo upload; PDF export with Puppeteer; auto-fill from applicant data	✓
Welfare Checklist	Admin-managed categories/items; drag-and-drop reorder; per-case Y/N/NA responses; completion tracking	✓
Workflow Stages	Admin-configurable stages; role/user permissions per stage; SLA configuration; reorder with drag-and-drop	✓
Dashboard & Analytics	Role-specific stat cards; case pipeline chart; recent activity feed; all counts filtered by user scope	✓
Notifications	In-app notifications for status changes, assignments, SLA alerts; mark read (individual / bulk); unread badge	✓
Master Data	CRUD for all 8 master tables; Jamiat/Jamaat import/export via Excel; executive level ordering	✓

20.2 System-Level Acceptance

CRITERIA	STATUS
All API routes require valid JWT authentication	✓
RBAC enforced on every route via permissionMiddleware	✓
Jamiat/Jamaat scope isolation applied to all non-admin data queries	✓
All DB queries use parameterized statements (no SQL injection risk)	✓
All file uploads validated for type and size before storage	✓
System accessible via HTTPS in production	✓
React SPA fully functional with JavaScript disabled gracefully redirecting to login	✓
PDF generation produces valid, downloadable PDF files for both cover letters and quarterly reports	✓

21. Glossary

TERM	DEFINITION
FACTS	File and Case Tracking System — the platform documented in this BRD
Baaseteen	Primary organization operating the FACTS system for welfare case management
SHND	Social Help and Development — the partner organization managing a subset of welfare cases
ITS	Institutional Information System — external identity registry providing applicant demographics via API lookup by ITS number
ITS Number	Unique 8-digit identifier assigned to each community member in the ITS registry; used as the primary applicant identifier in FACTS
Jamiat	Parent organizational unit (e.g., a city or regional body); cases are scoped to a Jamiat
Jamaat	Sub-organizational unit within a Jamiat (e.g., a local chapter); used for granular access scoping
DCM	Deputy Counseling Manager — role responsible for case assignment, counselor coordination, and resubmission to welfare
ZI	Zonal Incharge — role responsible for first-level executive approval after welfare review
Manzoori	Formal sanction/approval stage in the workflow; the Counseling Form section 8 (Manzoori) captures this approval
Enayat	Non-repayable financial grant component of welfare assistance
Qardan Hasana (QH)	Interest-free loan component of welfare assistance; subject to a repayment schedule
Rahen	Collateral or security offered by the applicant in support of a Qardan Hasana request
Cover Letter	Formal case summary document generated as a PDF at the Cover Letter stage of the workflow, capturing financial summaries and recommendations
Fund Utilization Report	Quarterly reassessment report tracking how disbursed funds are being used; submitted per case per quarter
Welfare Checklist	Structured evaluation checklist completed by welfare reviewers to assess case file completeness
SLA	Service Level Agreement — maximum time allowed at each workflow stage before escalation; tracked per case as on_time / at_risk / breached
RBAC	Role-Based Access Control — permission model where access is granted via resource:action pairs assigned to roles
JWT	JSON Web Token — signed authentication token issued on login; expires after 24 hours
Counseling Form	The main 8-section digital form capturing all welfare case assessment data
Case Identification	Pre-case eligibility screening record; reviewed as eligible (creates a case) or ineligible
Business Assets	6-year financial projection table (8 asset/liability categories × last year + 5 projected years) linked to a case
Workflow Stage	A configurable step in the case approval pipeline, with associated roles, permissions, and SLA settings
is_duplicate flag	A flag set on a case when the same ITS number has more than one active case; surfaced in UI for admin review
PDFKit	Node.js library used for programmatic PDF generation (cover letters)
Puppeteer	Headless Chromium browser used to render HTML templates as PDF files (quarterly reports)

22. Appendices

Appendix A: API Endpoint Summary by Module

MODULE	BASE URL	KEY ENDPOINTS	COUNT
Authentication	`/api/auth`	login, register, profile, change-password, logout	6
Users	`/api/users`	CRUD, toggle-status, assign-executive-level, import-excel, fetch-from-api	16
Cases	`/api/cases`	CRUD, workflow actions (welfare/ZI/executive), comments, payment-schedule, close, closure	27
Case Identifications	`/api/case-identifications`	List, get, create, review, edit, revert-to-pending, check-its	7
Applicants	`/api/applicants`	CRUD, ITS lookup, fetch-from-api, bulk-import, bulk-fetch, import-excel, export-template	13
Counseling Forms	`/api/counseling-forms`	Get form, update section, mark complete	3
Cover Letters	`/api/cover-letters`	Generate PDF, get for case, download	3
Cover Letter Forms	`/api/cover-letter-forms`	Get, create/update, submit	3
Dashboard	`/api/dashboard`	Overview, recent-activities, case-pipeline	3
Notifications	`/api/notifications`	List, mark-read, mark-all-read, unread-count, delete, check-sla	6
Attachments	`/api/attachments`	Upload (single/multiple), list, download, delete, stats	6
Roles	`/api/roles`	CRUD, available-permissions, assign, unassign, stats	10
Permissions	`/api/permissions`	Check, counseling-form-access, counseling-form-roles, admin-roles, summary	5
Workflow Stages	`/api/workflow-stages`	CRUD, restore, reorder, manage roles/users per stage, status-mappings	16
Welfare Checklist	`/api/welfare-checklist`	Category CRUD+reorder, Item CRUD+grouped, responses, status	14
Fund Utilization	`/api/cases/:id/fund-utilization`	CRUD reports, upload photos/files, delete photos/files	9
Fund Utilization Master	`/api/fund-utilization-master`	Form-config, sections CRUD+reorder, questions CRUD+reorder	11
Quarterly Report PDF	`/api/cases/:id/quarterly-report`	Generate and download PDF	1
Jamiat / Jamaat	`/api/jamiat`, `/api/jamaat`	CRUD, import, export, template, by-jamiat filter	14
Master Data (5 tables)	`/api/case-types` etc.	CRUD per table; executive-levels adds reorder + active list	25+
Business Assets	`/api/business-assets`	Get, create/update, delete per case	3

Total: 150+ endpoints across 25 route files

Appendix B: Backend File Structure

PATH	PURPOSE
`backend/index.js`	Express server entry point; mounts all routers; configures middleware stack

PATH	PURPOSE
`backend/config/database.js`	MySQL connection pool (10 connections) via mysql2
`backend/middleware/auth.js`	JWT verification, RBAC enforcement, Jamiat/Jamaat scope injection
`backend/middleware/permissionMiddleware.js`	Helper functions for permission checking patterns
`backend/routes/`	25 route files (one per module domain)
`backend/services/emailService.js`	Nodemailer SMTP email sending with template support
`backend/services/notificationService.js`	In-app notification creation for case events
`backend/utils/slaCalculator.js`	SLA status calculation (on_time / at_risk / breached) per case/stage
`backend/utils/activeCasePerIts.js`	One-active-case-per-ITS enforcement utility
`backend/utils/permissionUtils.js`	Permission helper functions used across routes
`backend/utils/roleUtils.js`	Role resolution and hierarchy utilities
`backend/utils/quarterlyReportPdfData.js`	Data aggregation for quarterly report PDF
`backend/utils/quarterlyReportPdfKit.js`	PDFKit-based report builder
`backend/templates/quarterlyReportPdfTemplate.js`	HTML template rendered by Puppeteer for quarterly PDF
`backend/uploads/`	Runtime file storage for case attachments, closure docs, fund utilization files/photos

Appendix C: Frontend File Structure

PATH	PURPOSE
`frontend/src/App.js`	React Router v6 configuration with all protected route guards
`frontend/src/contexts/AuthContext.js`	Auth state (JWT token, user object, permissions) via React Context
`frontend/src/pages/`	31 page components (one per major screen)
`frontend/src/components/ui/`	16 reusable UI components (Button, Card, Modal, Table, Alert, Badge, Input, Select, SearchableSelect, MultiSelect, Pagination, Toast, Tabs, Spinner, WorkflowProgress)
`frontend/src/components/CounselingForm.js`	8-section tabbed counseling form with section-level save
`frontend/src/components/CoverLetterForm.js`	Cover letter form with pre-populated applicant/financial data
`frontend/src/components/PaymentSchedule.js`	Payment schedule builder and disbursement confirmation
`frontend/src/components/SignaturePad.js`	Digital signature component (draw or file upload modes)
`frontend/src/components/CaseSLAStatus.js`	SLA status badge and countdown display
`frontend/src/utils/permissionUtils.js`	`usePermission` hook for component-level permission checks
`frontend/src/utils/generateCoverLetterPDF.js`	Client-side cover letter PDF generation helper
`frontend/build/`	Production build output served by Nginx

Appendix D: Database Schema Files

FILE	PURPOSE
`database/baaseteen_production_database.sql`	Consolidated production schema — single file for full DB import
`database/migrations/create_case_identification_income_logs_table.sql`	Income change audit log table
`database/migrations/create_fund_utilization_tables.sql`	Fund utilization report, photos, and files tables
`database/migrations/create_fund_utilization_master_tables.sql`	Configurable form builder sections and questions tables
`database/migrations/alter_fund_utilization_for_form_builder.sql`	Schema update for dynamic form builder support
`database/migrations/seed_fund_utilization_sections_and_questions.sql`	Default sections and questions for fund utilization form

Appendix E: Environment Variables Reference

VARIABLE	REQUIRED	DESCRIPTION
`DB_HOST`	Yes	MySQL host (e.g., `localhost`)
`DB_USER`	Yes	MySQL username
`DB_PASSWORD`	Yes	MySQL password
`DB_NAME`	Yes	Database name (`baaseteen_cms`)
`JWT_SECRET`	Yes	JWT signing key (minimum 32 characters; use a cryptographically random value)
`JWT_EXPIRES_IN`	Yes	Token expiry (e.g., `24h`)
`PORT`	Yes	Express server port (default: `5000`)
`NODE_ENV`	Yes	`development` or `production`
`FRONTEND_URL`	Yes	React app origin for CORS (e.g., `https://your-domain.com`)
`EMAIL_HOST`	No	SMTP host (e.g., `smtp.gmail.com`)
`EMAIL_PORT`	No	SMTP port (e.g., `587`)
`EMAIL_USER`	No	SMTP username / email address
`EMAIL_PASS`	No	SMTP password or app-specific password
`EMAIL_FROM`	No	From address for sent emails
`MAX_FILE_SIZE`	No	Max upload file size in bytes (default: `10485760` = 10 MB)
`UPLOAD_PATH`	No	File upload directory (default: `./uploads`)
`REACT_APP_API_URL`	Yes	Backend API base URL used by frontend (e.g., `http://localhost:5000`)
`REACT_APP_ENV`	No	`development` or `production`
`LOGIN_RATE_LIMIT_SKIP_IPS`	No	Comma-separated IPs excluded from login rate limiting (e.g., internal monitoring)

— End of Business Requirements Document —

FACTS Case Management System
Version 3.7 — April 23, 2025